Streams and Formatting Input-Output Operations in C++

Stream

- A stream is a conceptual pipe-like structure, which can have one end attached to the program and the other end attached by default to a keyboard, screen, or file. It is possible to change where one end is pointing to, while keeping the other end as it is

I/O STREAMS OF C VS C++

- Due to the backward compatibility needed with C, C++ provides C stream I/O as it is. One can still include <stdio.h> and use printf() and scanf() in the programs (though it is not recommended)
- It also provides a new I/O (the one with cout, cin, and << and >> operators overloaded). The new I/O is also stream-based.

Distinct advantages of using C++ I/O over C I/O

- Objects represent streams in C++
  - cout and cin is an object of output and input stream classes
  - Using objects from istream or ostream class enables the users to overload operators provided by that stream to use it for user-defined types

Cont…

- C++ I/O stream contains richer formatting options than C. It is possible to have the programmer's own format operators known as user-defi ned manipulators in C++
- C++ I/O is much easier to use

- So we can say that **C++ I/O streams are object-oriented, contains richer formatting options, and are more user-friendly than C-type I/O.**

PREDEFINED AND WIDE CHARACTER STREAMS

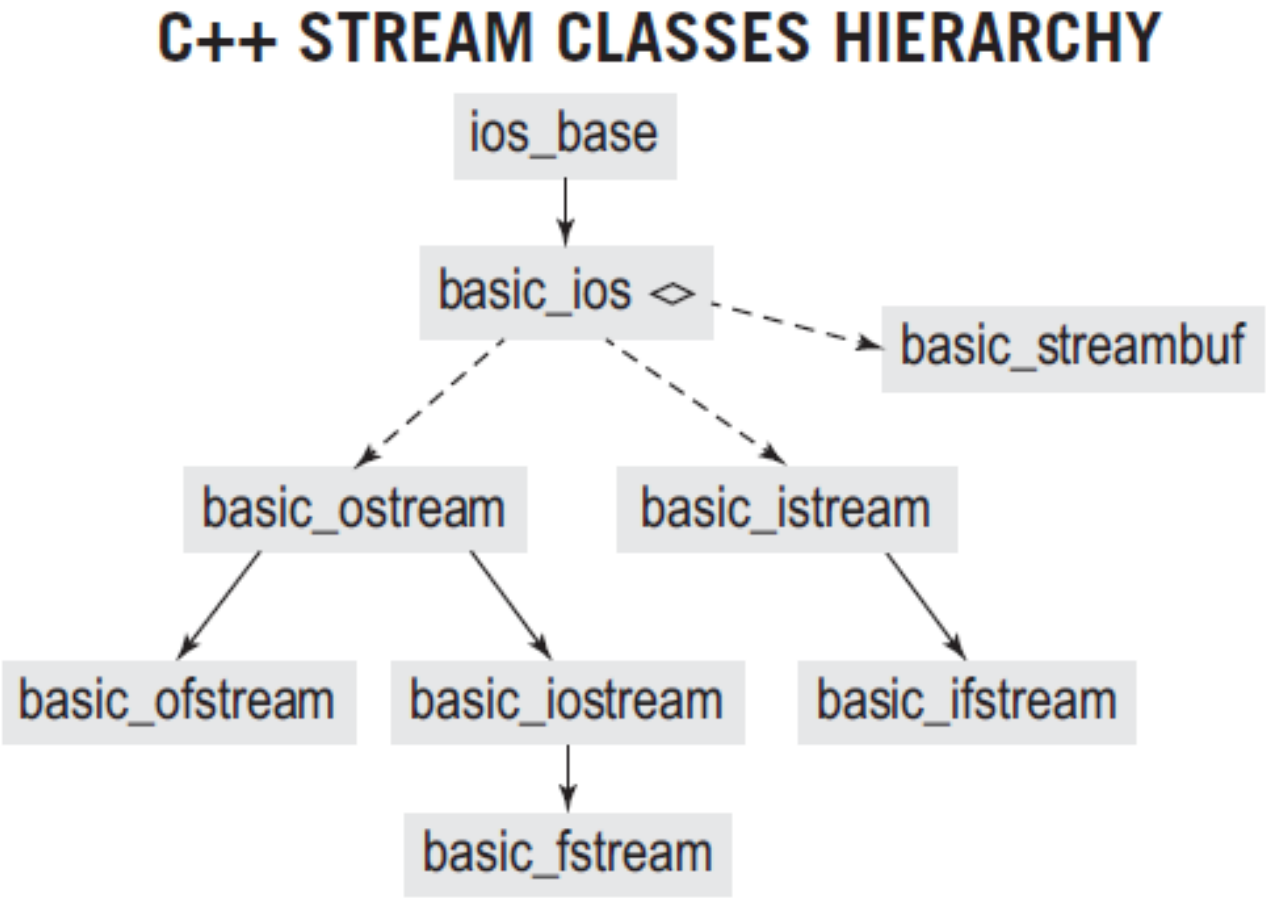
Predefined streams

Stream	Meaning	Default source/destination
cin	Standard input stream	Keyboard
cout	Standard output stream	Screen
cerr	Standard error stream with no buffer	Screen
clog	Standard error stream with buffer	Screen

PREDEFINED AND WIDE CHARACTER STREAMS :

`win`, `wout`, `werr`, and `wlog`

C++ STREAM CLASSES HIERARCHY



I/O hierarchy

UNFORMATTED I/O : get and put

```
#include <iostream>
using namespace std;
int main()
{
char ch;
char Data1Read[100];
char Data2Read[100];
for(int i = 0; ; i++)
{
cin.get(ch);
if(ch == '\n')
{
Data1Read[i] = '\0';
break;
}
Data1Read[i] = ch;
}
for(i = 0; ch = Data1Read[i]; i++)
{
cout.put(ch);
}
cout.put('\n');
```

```
for(int i = 0; ; i++)
{
ch = cin.get();
// Other version of get()
if(ch == '\n')
{
Data2Read[i] = '\0';
break;
}
Data2Read[i] = ch;
}
for(i = 0; ch = Data2Read[i];
i++)
{
cout.put(ch);
}
cout.put('\n');
}
```

Output

```
This is testing of first get and
put
This is testing of first get and
put
This is testing of second get
and put
This is testing of second get
and put
```

getline(), read(), and write() Functions

- cin.getline(string variable, maximum size of string that can be input)
- cin.read(string variable, maximum size of string that can be input)
- cout.write(string variable, maximum size of string that can be output)

```
#include <iostream>
using namespace std;
int main()
{
char Message[100];
cin.getline(Message, 100);
cout.write(Message, 50);
cout << endl;
cin.read(Message, 100);
cout.write(Message, 50);
cout.write("\n", 1) ;
}
```

Output

```
This is testing of getline and then we
have read test
This is testing of getline and then we
have read t
Now the read is being tested
It requires ctrl-Z to end
^Z
Now the read is being tested
It requires ctrl-Z to
```

- FORMATTING I/O

Formatting using ios functions

Function	Default	Effect
width()	0	Minimum field with to be used
precision()	6	Precision for number of digits after decimal point
fill()	Space character	Fill character to be used for padding
setf()	0	Set bit of format flag
unsetf()	0	Reset bit of format flag

width()

- Function width() is used to get/set width variable. Width specifies number of characters or positions to be used during input/output. Even though it can be used during input, it is more meaningful to use it for output purpose.
- Syntax: int ios::width (int num=0)

width()

- For example:
  - int i=1234; str[10]=”Hello”;
  - cout.width(7);
  - cout<<i << str;

- In the above example, value of the first item i will be displayed using a width of 7 positions and the width parameter will be reset to its default value 0. Now, value of str will be displayed using minimum width 0; i.e 5 positions as necessary.

width()

- cin.width(7); cin >> i;
- cin.width(7); cin>> str;

- Let the input be: 1234GujaratHello
- Here, extraction for integer i is independent of width parameter. Characters 1234 are extracted and transformed to binary number to be stored in variable i. Next, 7 characters will be extracted and stored in variable str.
- Thus specified width will be applied first time to item str and reset to 0.

fill()

- Function fill() is used to get/set fill character used as padding character to fill extra positions.
- Syntax: char ios::fill (char padding = ‘ ’)
  - cout.width(7);
  - cout.fill (‘*’);
  - cout << 1234;
  - cout.width(7);
  - cout << “Hello”;
  - Output will be: ***1234**Hello

precision()

- This function is used to specify number of significant digits in case of floating point numbers.

- Syntax: int ios::precision (int num = 6)

- For example:
  - cout.fill ('*');
  - cout.precision(7);
  - cout.width(10);
  - cout << 1234.567<< endl;
  - cout.precision(8);
  - cout.width(10);
  - cout << 1234.567<< endl;
  - cout.precision(5);
  - cout.width(10);
  - cout << 1234.567<< endl;
  - cout.precision(3);
  - cout.width(10);
  - cout << 1234.567<< endl;
  - **Output is:**
  - ****1234.567**
  - ****1234.567**
  - ******1234.6**
  - ****1.23e+03**

Format flags

Flag name	Description
skipws	Skip initial white space while reading from a stream.
left	Output is left justified.
right	Output is right justified.
internal	Numeric value is padded with space between sign or base (0x, 0, etc.) character.
oct	Output is displayed in octal format.
hex	Output is displayed in hexadecimal format.
dec	Output is displayed in decimal format.
showbase	Numeric values will be displayed with base character such as displaying Hex 1E as 0x1E (Hex also needs to be specified before this, because decimal is the default).
uppercase	The x in showbase and the e in scientific notation are displayed in lower case. If uppercase is set, they would be displayed as X and E, that is, in upper case.
showpos	A leading + sign is displayed before positive values. This is not displayed by default.
showpoint	Decimal point will be displayed even if it is not present. Trailing zeros will also be displayed for all floating point values (this is needed for the example given earlier).
scientific	Output displays floating point values in scientific notation.
fixed	Output is displayed in fixed (non-scientific) notation.
unibuf	Buffers are flushed after every insertion. It is similar to flushing a stream after reading, so that no more extra chars are left out and subsequent read statements have no problem.
boolalpha	Enable the Boolean values or the input to be displayed as true or false rather than 1 and 0.
basefield	Collection of oct, dec, and hex. Needs to be specified as the second argument if any one of the three is the first argument to `setf()`
adjustfield	Collection of left, right, and internal field. Needs to be specified as the second argument if any one of the three is the first argument to `setf()`
floatfield	Collection of scientific and fixed. Needs to be specified as the second argument if any one of the two is specified as the first argument

setf()

- Format flags (bits of format flag variable) can be set using setf() function in two forms as follows.
- Syntax:
  - fmtflags ios::setf (fmtflags flg)
  - **Flag to set :**
    - left, right, internal
    - dec, oct, hex
    - scientific, fixed

Setf flags example

- Printing mark sheet

```
#include <iostream>
using namespace std;
int main()
{
cout.width(15);
cout.setf(ios::left); cout.setf(ios::fixed); cout.precision(2);
cout.setf(ios::showpoint);
/* Code for printing */
}
```

Output

Roll number	Name	Marks
1	Lara	355.50
2	Beckham	275.00
3	Steffi	290.75
4	Jaspal	295.00
5	Ranatunga	200.60

unsetf()

- Format flags can be reset using unsetf() function.
- Syntax:
- fmtflags ios::unsetf (fmtflags flg)

Formatting using manipulators

- Like member functions of ios class, manipulators are also used to perform the task of formatting input/output. Manipulators are special type of non-member functions that can be included in an insertion or extraction chain.
- Manipulators are non-member functions returning a reference to a stream. So they can be used with shift operator

Manipulator

- setw()
- setprecision()
- setfill()
- setiosflags()
- resetiosflags()

- Note : To use manipulators, it requires to include <iomanip> in C++ program.

- cout<<“Numbers left justified in width 10 : ” << endl;
- cout.setf(ios_base::left,ios_base::adjustfield);
- cout.width(10);
- cout<<12;
- cout.width(10);
- cout<<123<< '\n';

- The above C++ statements can be written in a single statement using manipulators as follows:
- cout<< “Numbers left justified in width 10 : ” << endl << left << setw(10) << 12 << setw(10) << 123

  << '\n' ;

User-defined manipulators

- The code structure for user-defined manipulator is as follows:
  - ostream &<manipulator_name> (ostream &<parameter_name>)
  - {
    - // code for formatting …
  - return <parameter_name>;
  - }

- For example, define manipulator km to print KiloMeter as follows:
  - ostream& km (ostream & pout) { pout << “ KiloMeter” ; return pout;}
  - Using cout << 200 << km; will output 200 KiloMeter

Using ios functions	Using manipulators
Functions are members of ios class	Manipulators are non-member functions
Require only <iostream> file to be included	Require <iomanip> file to be included additionally
Return the previous status of format parameters like width, precision, fill character and format Flags	Does not return previous status of format parameters, Return a reference to stream
Used as a standalone statement, ex. cout.width(10); cout.fill('*');	Can be cascaded with shift operator, ex. cout << setw(10) << setfill('*');
For some manipulators, there is no equivalent function available, for example, endl	Manipulators available corresponding to all ios functions
No shorthand names to use	Can use shorthand names
Not possible to define user-defined member functions in ios class, one may create another class inheriting from ios and add user-defined functions	User-defines manipulators can be easily created
Function calls always requires use of (), example: int w = width();	Manipulators without argument can be called without (), example: endl